



Seminario de Programación



C++ - Manejo de Memoria y Debugging

Dr. Nicolás Gálvez Ramírez

Ingeniería Civil Telemática
Departamento de Electrónica
Universidad Técnica Federico Santa María

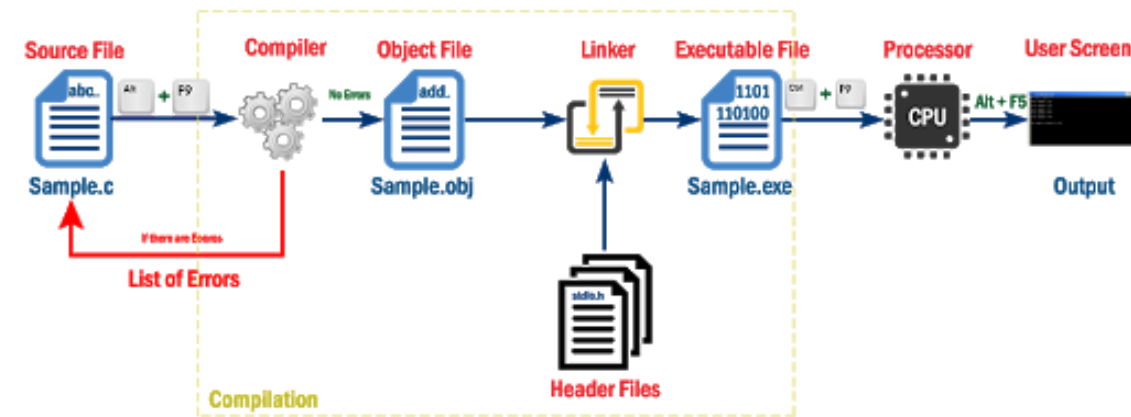
Etapas de ejecución

Etapas de ejecución

- **Ejecutar:** llevar un programa (binario) a memoria primaria (RAM, Caché, Virtual, swap).
- **Variables y Constantes:** segmentos de que se asignan y liberan en memoria.
- **Ciclo de vida:** cuando se crea y destruye una variable.
- **Ámbito:** lugar donde ocurre el Ciclo de Vida, vale decir, porción de código que tiene acceso a una variable.

Etapas de ejecución

- **Ejecutar:** llevar un programa (binario) a memoria primaria (RAM, Caché, Virtual, swap).
- **Variables y Constantes:** segmentos de que se asignan y liberan en memoria.
- **Ciclo de vida:** cuando se crea y destruye una variable.
- **Ámbito:** lugar donde ocurre el Ciclo de Vida, vale decir, porción de código que tiene acceso a una variable.



Ámbito

Ámbito

Antes de desglosar como funciona la memoria en C/C++, es necesario entender el concepto de Ámbito.

- **Ambito:** Lugar donde ocurre el Ciclo de Vida, vale decir, porción de código donde se tiene acceso a una variable.
- Se delimita por el uso de las llaves de conjunto `{` y `}`.
- Nos ayudan a definir variables globales y locales.

Ámbito

Antes de desglosar como funciona la memoria en C/C++, es necesario entender el concepto de Ámbito.

- **Ambito:** Lugar donde ocurre el Ciclo de Vida, vale decir, porción de código donde se tiene acceso a una variable.
- Se delimita por el uso de las llaves de conjunto { y }.
- Nos ayudan a definir variables globales y locales.

```
In [ ]: #include <iostream>

int main(){
    int a; // Acá se crea a
    {
        float b; //Acá se crea b
        std::cout << a << b << std::endl; //válido
    } // Acá se destruye b

    std::cout << a << b << std::endl; //inválido
```

```
    return 0;  
} // Acá se destruye a
```

Variables Globales y Locales

Variables Globales y Locales

- **Variables Globales:** Aquellas variables definidas fuera de cualquier ámbito. Son visibles y utilizables por cualquier función o estructura de control, si no hay una variable local con el mismo nombre.
- **Variables locales:** Aquellas variables definidas dentro de un ámbito específico. Son visibles ese ámbito y sus ámbitos internos.

Nota: A diferencia de Python, en C/C++ las variables globales pueden ser editadas dentro de un ámbito local. Esto debido a la diferencia de declaración de variables (Python → implícita, C/C++ → explícita).

```
In [ ]: #include <iostream>

int cont = 1; // variable global

int le_funcion(int a){ //parámetros -> variables locales
    int cont = a; // variable local
    cont++;

    return cont;
}

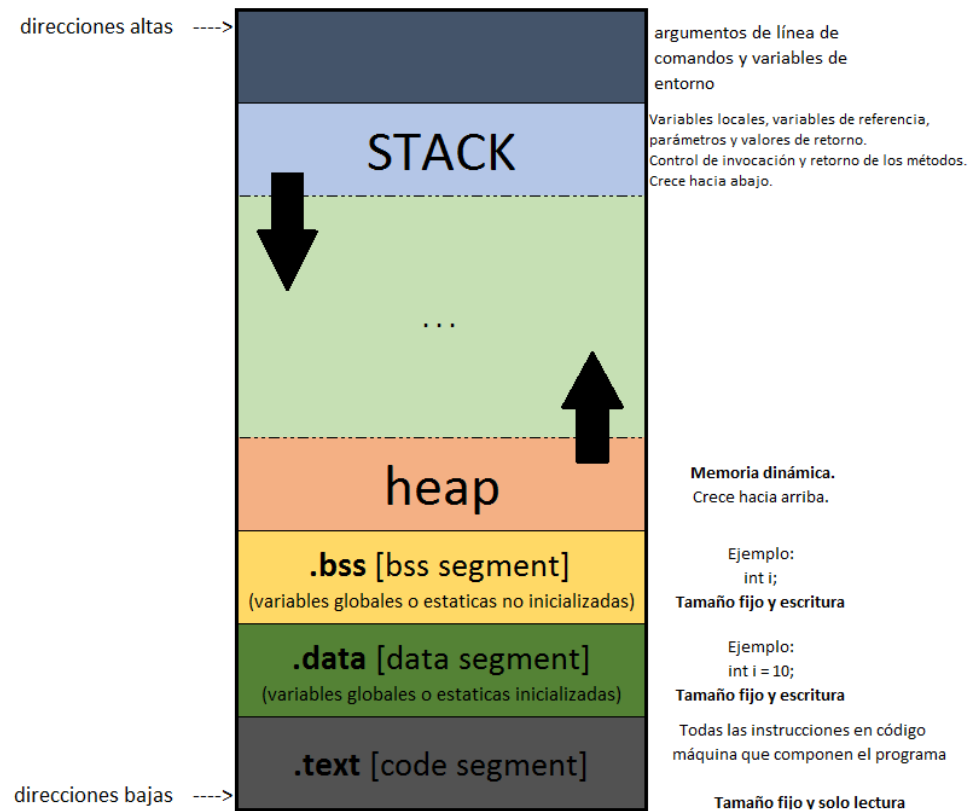
int main(){
    int a = 4; // variable local
    if(a==5){
        cont++;
    }else{
        int cont = le_funcion(a);
    }
    std::cout << cont << std::endl;

    return 0;
}
```

Clasificación de la Memoria en C/C++

Clasificación de la Memoria en C/C++

- **Segmento de código o texto (code):** donde el programa compilado se mantiene en memoria.
- **Segmento estático (static):** almacena variables globales y estáticas, incluyendo el nombre de funciones. Realiza ligado estático de variables en la memoria (hasta que programa termina). Se divide en dos segmentos, para las variables declaradas, y otro para variables inicializadas.
- **Segmento o memoria heap:** Segmento donde se le permite **al programador** almacenar objetos de memoria de forma dinámica. Son utilizados indirectamente mediante una variable como un **puntero** o una referencia. **Depende del programador.**
- **Segmento o memoria stack:** área dinámica de memoria donde se guardan los parámetros de las funciones, información de las funciones, variables y objetos que se asignan y liberan **automáticamente por el compilador.**



Segmento de Código
(code)

¿Cómo se almacena en `code`?

¿Cómo se almacena en **code**?

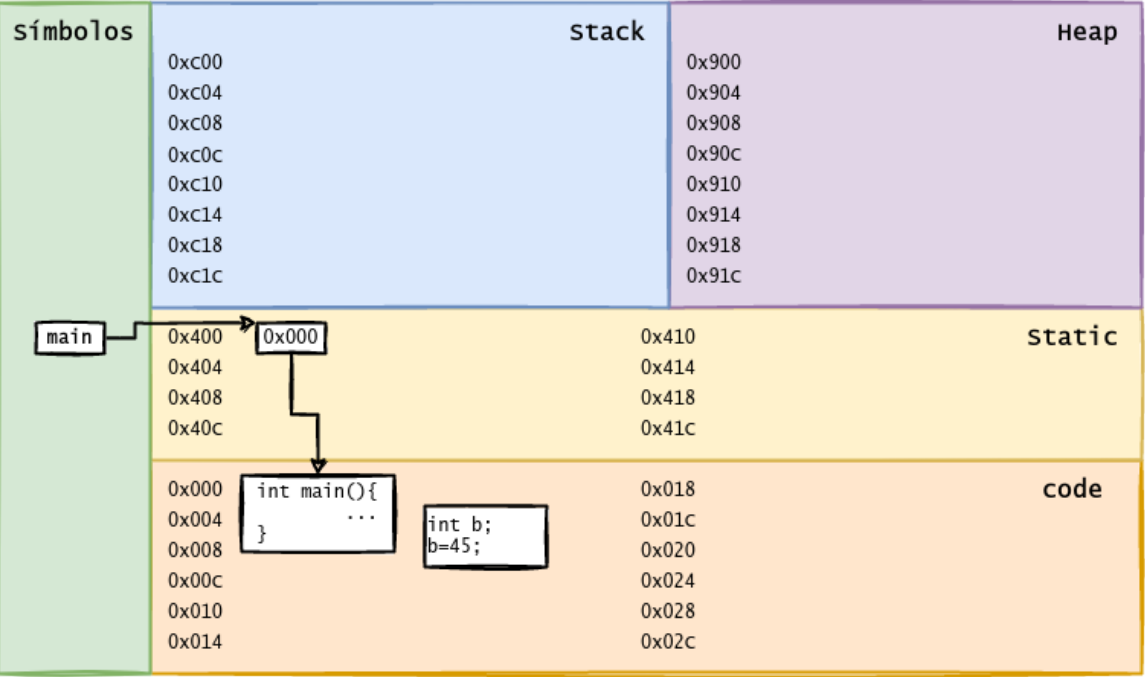
```
In [ ]: #include <iostream>

int main(){

    int b;
    b = 45;

    std::cout << "B es " << b << std::endl;

    return 0;
}
```



Segmento Estático
(static)

Variables estáticas

Variables estáticas

- Ligado a la memoria estática.
- El ligado permanece durante todo el tiempo de ejecución del programa.
- **Ventajas:**
 - Útil para definir variables globales y compartidas.
 - Para variables sensibles a la historia de un subprograma (e.g. modificador static).
 - Acceso directo permite mayor eficiencia.
- **Desventajas:**
 - Impide compartir memoria entre diferentes variables (no se reasigna).

```
In [ ]: #include <iostream>

long y; //visible desde todos los archivos del proyecto.

static int z = 1; //static solo cambia la visibilidad;
                //solo a este archivo.

extern float pi; //no se crea, sino que accede a var
                //global (no static) en otro archivo.

const float val = 3.14159; // variable global que no puede ser modificad

int fun(){
    int x = 0;
    static int y=0; //static va a memoria estática.
    x = y++;
    return x;
}
```


¿Cómo se almacena en `static`?

¿Cómo se almacena en `static`?

```
In [ ]: #include <iostream>

float pi = 3.1415;

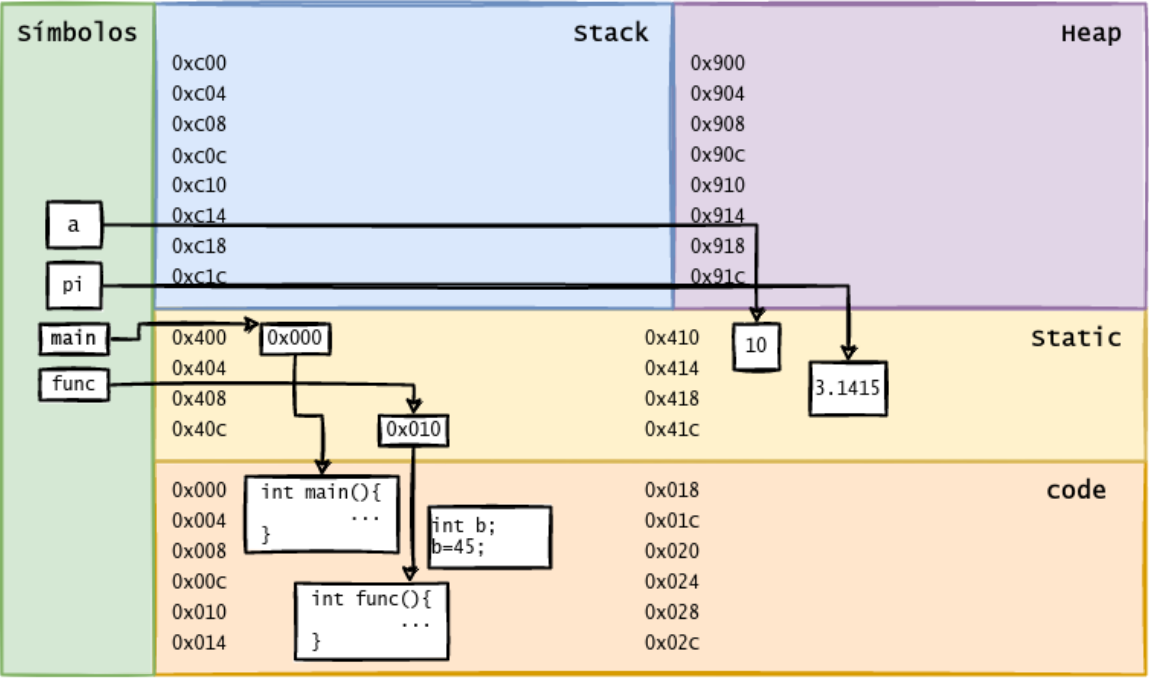
int func(){
    return pi;
}

int main(){
    static int a = 10;

    int b;
    b = 45;

    std::cout << "a es " << a << "y B es " << b << std::endl;

    return 0;
}
```



Segmento o Memoria Stack
(stack)

Segmento o Memoria Stack (stack)

- Se asigna memoria dinámicamente al compilar.
- Se libera cuando termina su ámbito de creación.
- El compilador se encarga de la administración (reservar y liberar) de la memoria. Automático.
- Soportan bien la recursión.

¿Cómo se almacena en `stack`?

¿Cómo se almacena en `stack`?

```
In [ ]: #include <iostream>

float pi = 3.1415;

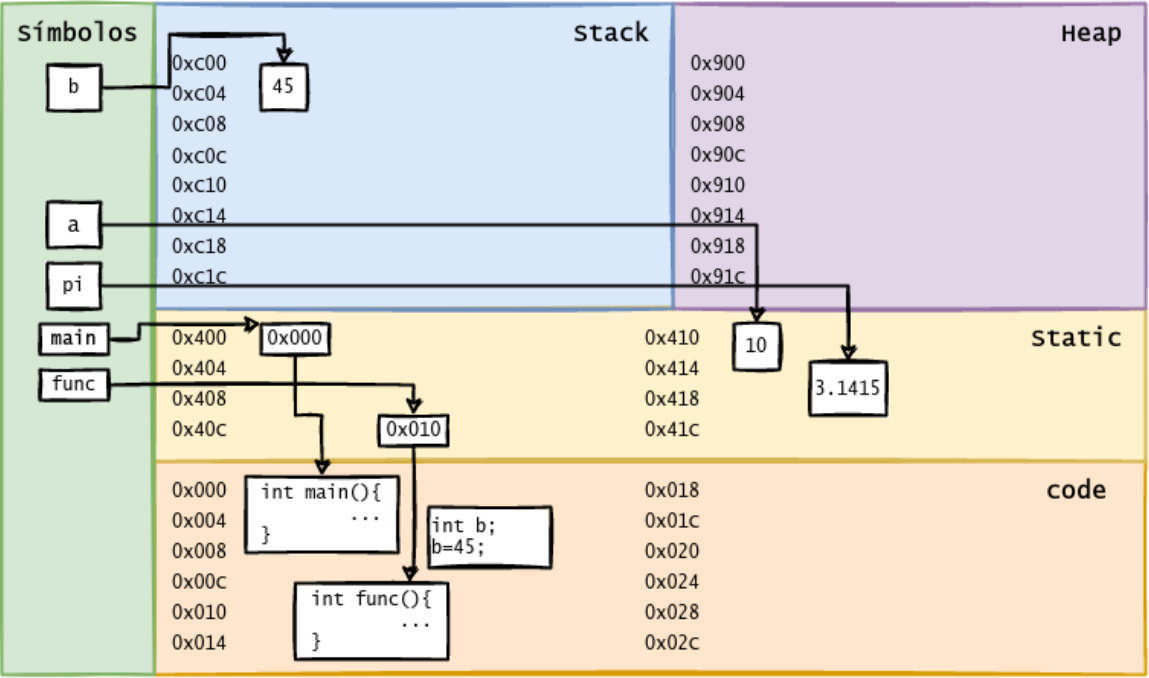
int func(){
    return pi;
}

int main(){
    static int a = 10;

    int b;
    b = 45;

    std::cout << "a es " << a << "y B es " << b << std::endl;

    return 0;
}
```



Segmento o Memoria Heap (heap)

Segmento o Memoria Heap (heap)

- Segmento de memoria el que es 100% dependiendo del programador.
- Es él quien debe asegurarse de la correcta implementación, reserva y liberación de los recursos de memoria a utilizar durante la ejecución del programa.
- Se realiza a través de la utilización de punteros y el almacenamiento de direcciones de memoria.

Punteros

Punteros

- Variables de stack que almacenan **direcciones de memoria** de un tipo de dato que está en memoria.
- Se pueden usar para hacer referencias indirectas al heap, stack, static y code.
- Nos centraremos en las dos primeras:
 - Heap → memoria dinámica.
 - Stack → paso por referencia.
- En *ELO-320* verán punteros para **direccionamiento** indirecto (listas, colas, pilas, árboles, etc).
- Un puntero se define como:

```
In [ ]: #include <iostream>

int main(){

    int val = 55; //variable val (stack) que contiene el valor 55.

    int * p;      // variable p (stack) que contiene la direccion de memoria de val
                  // o simplemente puntero a int.

    return 0;
}
```

- Los punteros se definen al igual que las variables, pero con el símbolo `*` adelante del nombre de la variable.
- Aprender a manejar memoria también nos introduce los operadores de referenciación de memoria:
 - **Derreferenciación (`*`)**: Accede a la dirección de memoria que almacena una variable (puntero).
 - **Referenciación (`&`)**: Obtiene la dirección de memoria dónde está creada una variable o elemento.
- Tomen en consideración que el operador `*` significa diferentes cosas según su uso:
 - **Creación de puntero**: en declaraciones o inicialización (lado izquierdo) de variable. `char *p; char *a = p;`
 - **Derreferenciación**: en asignaciones o inialización (lado derecho) de variable; `*p = 'a'; char b = *p;`


```
In [ ]: #include <iostream>

int main(){

    int val = 55; //variable val (stack) que contiene el valor 55.

    int *p = &val; //variable p (stack) contiene la dir. de memoria de val

    std::cout << "El valor de p (dirección guardada en stack): " << p << endl;
    std::cout << "La dir de memoria de val: " << &val << std::endl;
    std::cout << "El valor en la dirección guardada en p: " << *p << std::endl;
    std::cout << "El valor de val: " << val << std::endl;
    std::cout << "La dirección de p: " << &p << std::endl;

    return 0;
}
```

Ventajas y Desventajas

Ventajas y Desventajas

Pros:

- Útil para estructuras de datos dinámicas usando punteros (e.g. listas, pilas, colas).
- Gran control de uso de memoria por parte del programador.

Contras:

- Más líneas de código en manejo de memoria.
- Dificultad de uso correcto y en la administración de la memoria:
segmentation faults, memory leaks.

Uso del heap (reserva y liberación)

- Una de las grandes ventajas de los punteros, es poder reservar (y liberar) memoria desde el `heap`.
- Para esto necesitamos conocer dos comandos esenciales:
 - **new**: permite reservar memoria en el heap para almacenar el valor de un tipo específico.
 - **delete**: permite liberar la memoria reservada con anterioridad.
- En C++ también se puede utilizar las instrucciones de C para esta tarea:
 - **malloc**: reserva de memoria.
 - **free**: liberación de memoria.

```
In [ ]: #include <iostream>

int main(){

    int *val = new int; //puntero que apunta a un int en el heap (reservado)
    *val = 55; // es necesario reservar memoria para poder hacer esto

    float *p = (float *) malloc(sizeof(float)); //Reserva, en C++ es obligatorio
    *p = 3.1415; // asignación

    std::cout << "El valor del puntero val: " << val << std::endl;
    std::cout << "El valor al que apunta val: " << *val << std::endl;
    std::cout << "La dir de memoria del puntero val: " << &val << std::endl;

    std::cout << "El valor del puntero p: " << p << std::endl;
    std::cout << "El valor al que apunta p: " << *p << std::endl;
    std::cout << "La dir de memoria del puntero p: " << &p << std::endl;

    delete val; //libera la memoria reservada con new
    free(p); //liberación de la memoria reservada con malloc

    return 0;
}
```

¿Cómo se almacena en heap?

¿Cómo se almacena en heap?

```
In [ ]: #include <iostream>

float pi = 3.1415;

void func(){
    int *c = NULL; //si no se inicializa con reserva, lo mejor es usar el new
    c = new int;
    *c = 100;

    std::cout << "El valor en el heap: " << *c << std::endl;
    std::cout << "La dirección en el puntero (en stack): " << c << std::endl;
    std::cout << "La dirección del puntero: " << &c << std::endl;

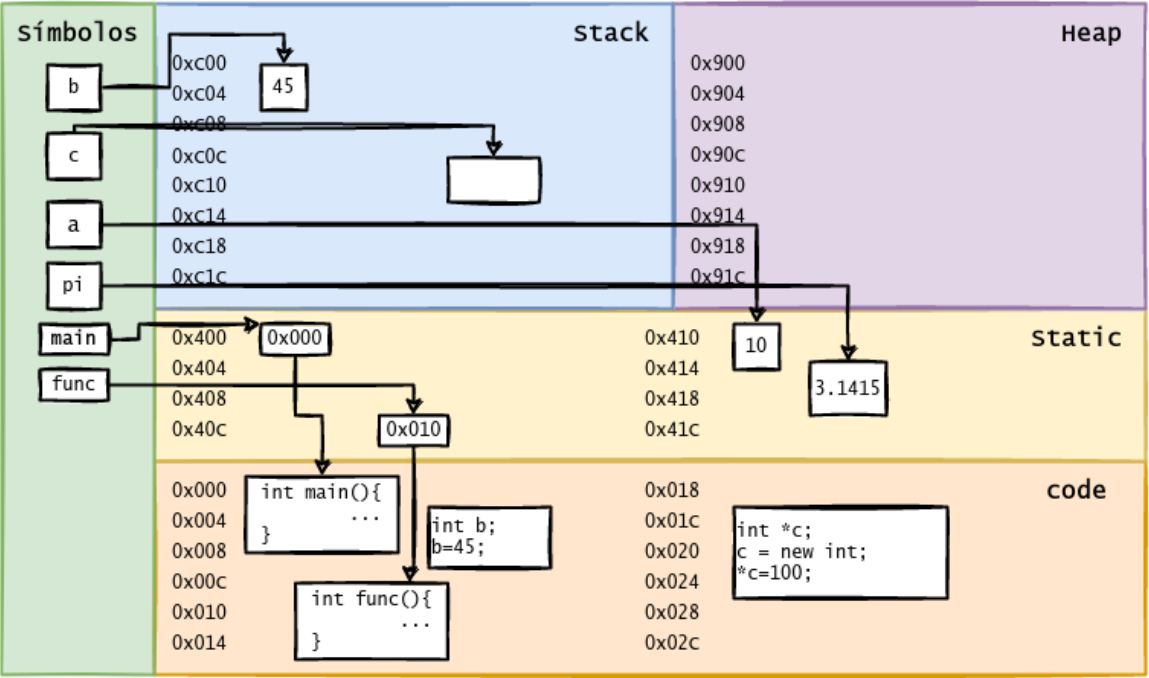
    delete c;
    c = NULL;
}

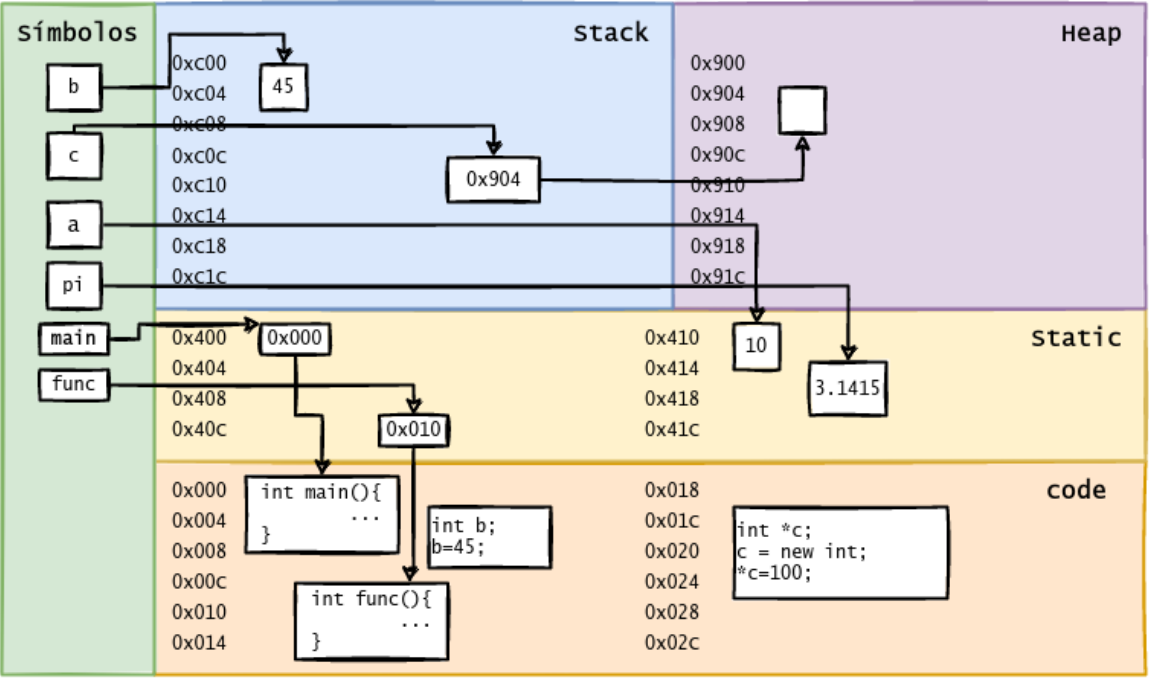
int main(){
    static int a = 10;

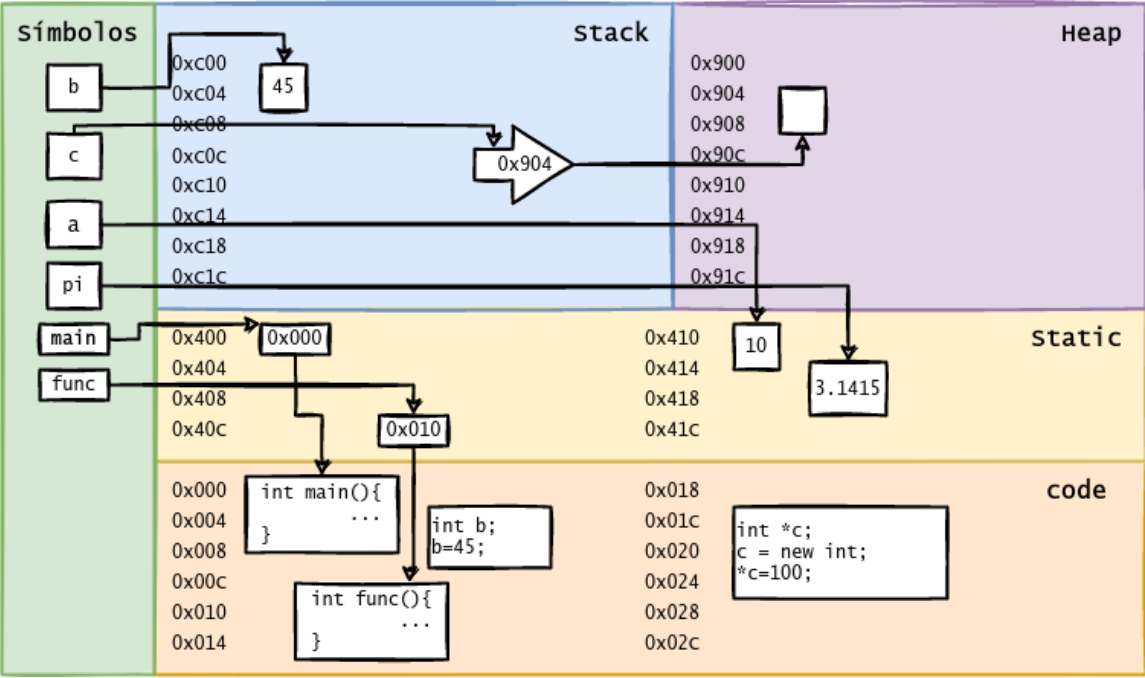
    int b;
    b = 45;

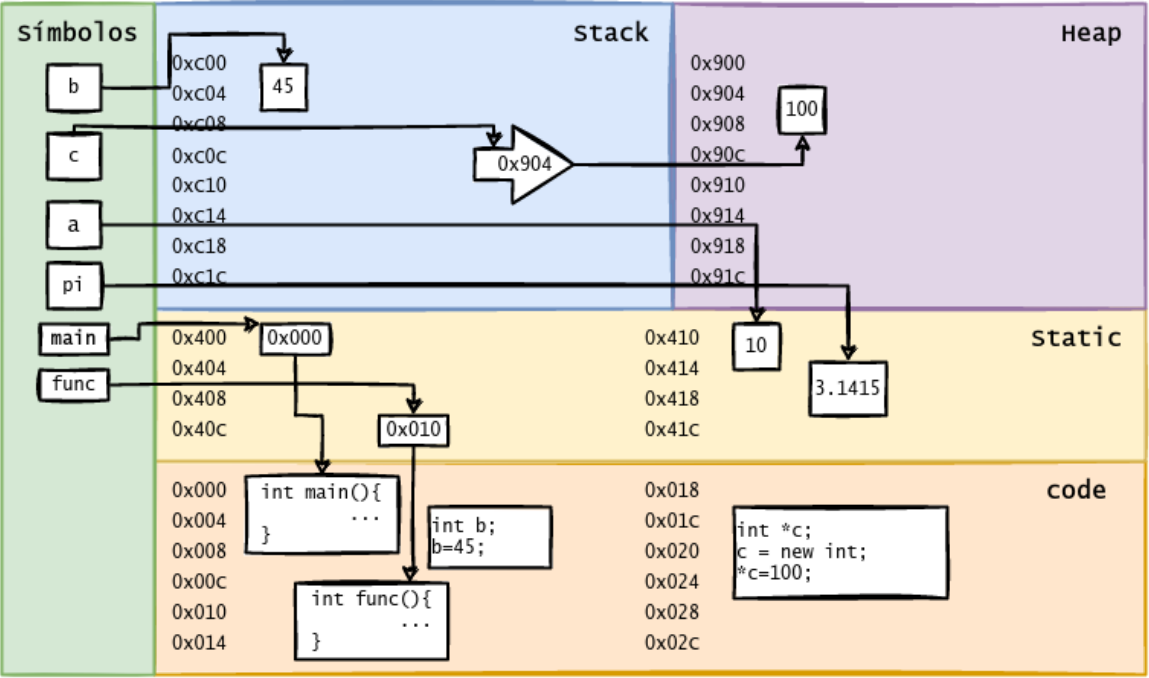
    func();
    std::cout << "a es " << a << "y B es " << b << std::endl;
```

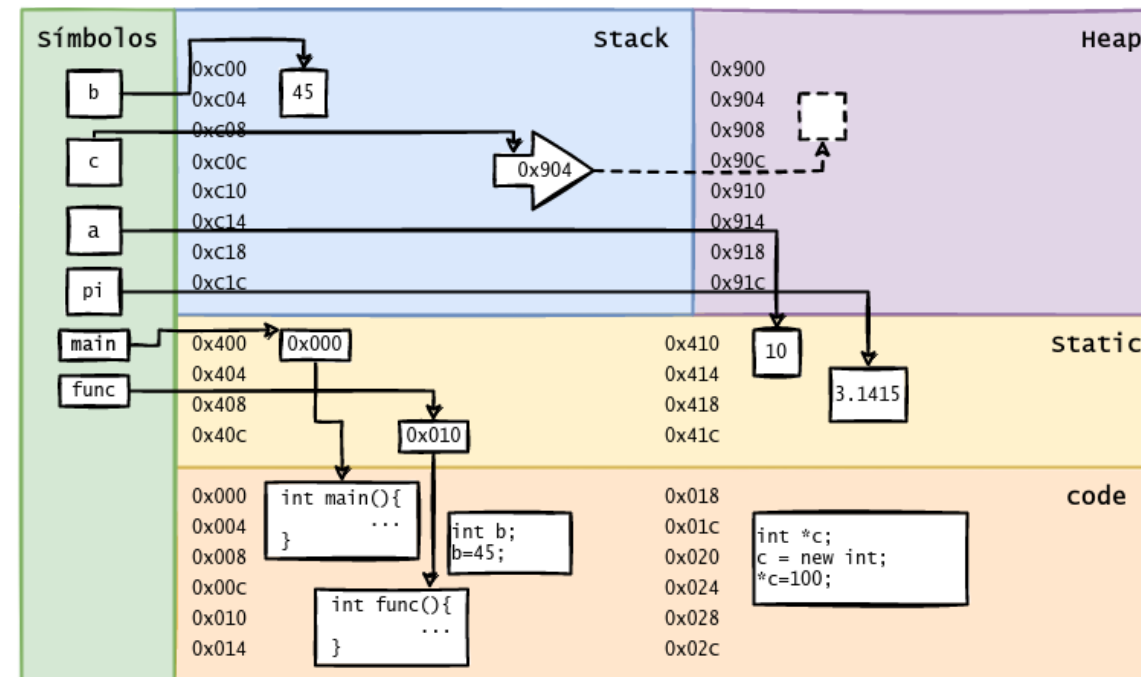
```
    return 0;  
}
```









Arreglos dinámicos y aritmética de punteros

Arreglos dinámicos y aritmética de punteros

Los arreglos pueden ser creados dinámicamente usando punteros.

- La reserva de memoria se hace por ≥ 2 unidades de un mismo tipo.
- El puntero guarda la dirección del primer elemento del arreglo.

Arreglos dinámicos y aritmética de punteros

Los arreglos pueden ser creados dinámicamente usando punteros.

- La reserva de memoria se hace por ≥ 2 unidades de un mismo tipo.
- El puntero guarda la dirección del primer elemento del arreglo.

```
In [ ]: #include <iostream>

int main(){

    int *p = new int[10]; //reserva 10 ints en el heap.
                          //(int *) malloc(sizeof(int)*10)

    for(int i=0;i<10;i++)
        p[i]=(i+1); //asignación como si fuese un arreglo.

    delete p;
```

```
return 0;
```

```
}
```

Un arreglo **no es** un puntero, sólo son **compatibles**. Pero, eso nos permite tratarlos de la misma forma.

Esto debido a que C/C++ realiza coerción (transformación automática) cuando hay alguna asignación. Esto se debe a que todo tipo de memoria también posee una dirección de memoria.

Un arreglo **no es** un puntero, sólo son **compatibles**. Pero, eso nos permite tratarlos de la misma forma.

Esto debido a que C/C++ realiza coerción (transformación automática) cuando hay alguna asignación. Esto se debe a que todo tipo de memoria también posee una dirección de memoria.

```
In [ ]: #include <iostream>

int main(){

    int a[10]; //reserva 10 ints en el stack.

    int *p = NULL; // puntero a int
    p = a; //esto C/C++ lo interpreta como p=&a[0]
           //vale decir la dirección de memoria del primer elemento de a

    for(int i=0;i<10;i++){
        p[i]=(i*i); //asignación como si fuese un arreglo.
        std::cout << a[i] << std::endl; //a se ve modificado.
    }
    //no es necesario liberar memoria -> a está en stack

    return 0;
}
```

Esta compatibilidad permite que la aritmética de punteros, sea también útil para manejar arreglos.

- Los punteros se suman en hexadecimal.
- `a[i]` es equivalente a `*(a+i)`.

Esta compatibilidad permite que la aritmética de punteros, sea también útil para manejar arreglos.

- Los punteros se suman en hexadecimal.
- `a[i]` es equivalente a `*(a+i)`.

```
In [ ]: #include <iostream>

int main(){

    int a[10]; //reserva 10 ints en el stack.

    int *p = NULL; // puntero a int
    p = a; //esto C/C++ lo interpreta como p=&a[0]
           //vale decir la dirección de memoria del primer elemento de a

    for(int i=0;i<10;i++){
        *(p+i)=(i+1); //asignación como si fuese un arreglo.
        std::cout << *(a+i) << std::endl; //a se ve modificado.
    }
    //no es necesario liberar memoria -> a está en stack

    return 0;
}
```

Funciones y punteros

Variable Referenciada

Variable Referenciada

En C++ existe otra forma de enlazar variables sin usar puntero, con variables referenciadas.

- Se usa el operador `&`, en la definición de la variable.
- Es inmutable una vez inicializada.

Variable Referenciada

En C++ existe otra forma de enlazar variables sin usar puntero, con variables referenciadas.

- Se usa el operador `&`, en la definición de la variable.
- Es inmutable una vez inicializada.

```
In [ ]: #include <iostream>

int main(){

    double a = 3.1415927;
    double &b = a; // b es a;
    b = 89;
    std::cout << "a: " << a << std::endl;

    return 0;

}
```

Paso por referencia en C++

Paso por referencia en C++

Las referencias pueden usarse para permitir a una función modificar a una variable pasada como parámetro.

Paso por referencia en C++

Las referencias pueden usarse para permitir a una función modificar a una variable pasada como parámetro.

```
In [ ]: #include <iostream>

void cambia(double &r, double s) {
    r = 100;
    s = 200;
}

int main() {
    double k = 3, m = 4;
    cambia(k, m);
    std::cout << k << ", " << m << std::endl; // Muestra 100, 4.

    return 0;
}
```


Las referencias pueden usarse también para que una función retorne una variable.

Las referencias pueden usarse también para que una función retorne una variable.

```
In [ ]: #include <iostream>

double &mayor(double &r, double &s) {
    if (r > s)
        return r;
    else
        return s;
}

int main () {
    double k = 3, m = 2;
    std::cout << "k: " << k << std::endl; // 3
    std::cout << "m: " << m << std::endl; // 2

    mayor(k, m) = 10;
    std::cout << "k: " << k << std::endl; // 10
    std::cout << "m: " << m << std::endl; // 2

    mayor(k, m)++;
    cout << "k: " << k << endl; // 11
    cout << "m: " << m << endl; // 2

    return 0;
}
```


Punteros y Struct

Punteros y Struct

Los punteros también pueden usarse para variables definidas a través de una estructura (struct). Para simplificar la notación de su uso, se introduce el operador `->`.

Punteros y Struct

Los punteros también pueden usarse para variables definidas a través de una estructura (struct). Para simplificar la notación de su uso, se introduce el operador `->`.

```
In [ ]: #include<iostream>
#include<cstring>

typedef struct character{
    char nom[30];
    char clase[30];
    int intel;
    int agi;
    int str;
    int mp;
    int def;
    int att;
} pj;

int main(){

    pj *player1 = new pj; //puntero a struct + reserva de memoria

    strcpy((*player1).nom, "Juanito"); // (*ptr).var equivale. ptr->var
    strcpy(player1->clase, "Warrior");
```

```
std::cout << player1->nom << " " << (*player1).clase << std::endl;
```

```
//¿Como añadirían valores al resto de los atributos?
```

```
delete player1;
```

```
return 0;
```

```
}
```


Errores Comunes y Debugging

Fugas de Memoria

Fugas de Memoria

Esta situación ocurre cuando se reserva memoria con un puntero que ya tenia asignada otro segmento.

- Se pierde la memoria anterior.
- Es imposible de recueprar.
- La memoria queda inutilizable hasta la finalización del programa.

Fugas de Memoria

Esta situación ocurre cuando se reserva memoria con un puntero que ya tenia asignada otro segmento.

- Se pierde la memoria anterior.
- Es imposible de recuperar.
- La memoria queda inutilizable hasta la finalización del programa.

```
In [ ]: #include <iostream>

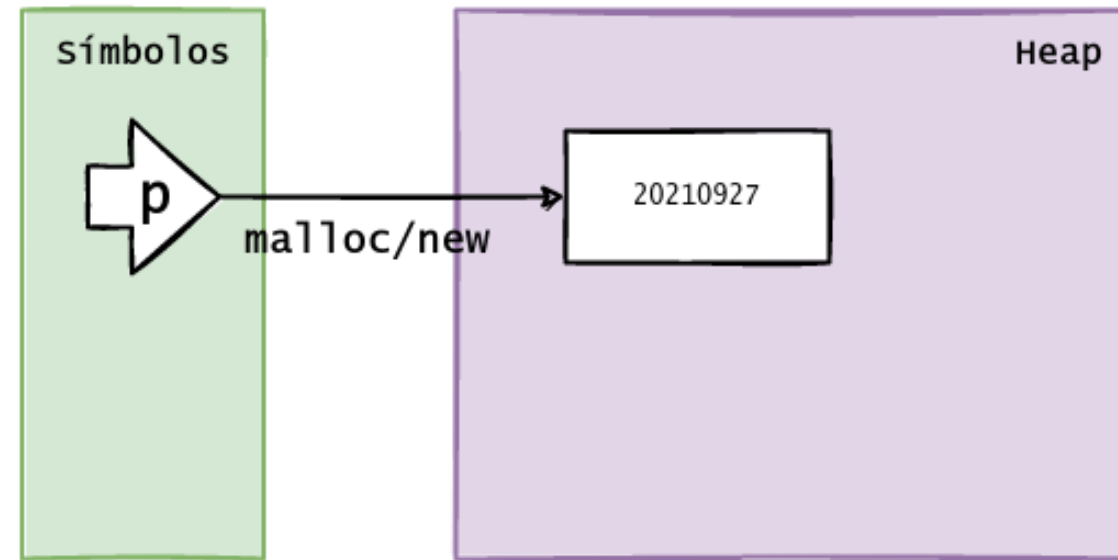
int main(){

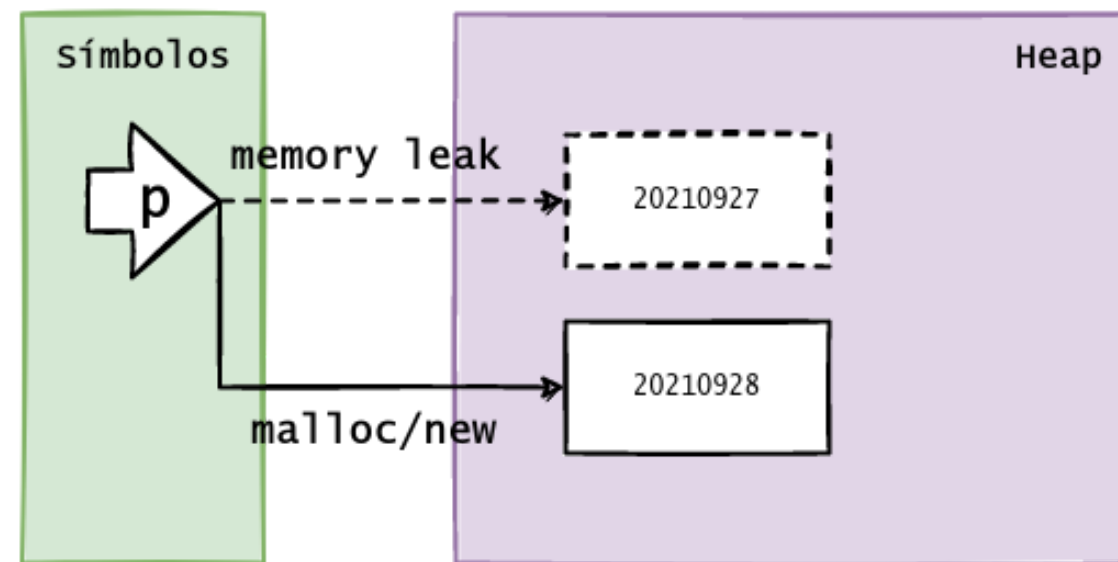
    int *p = new int;
    *p = 20210927;

    *p = new int;
    *p = 20210928;

    std::cout << "valor al que apunta p: " << *p << std::endl;

    return 0;
}
```





Dangling (dejar colgado)

Dangling (dejar colgado)

Esta situación ocurre cuando un segmento de memoria es referenciado por más de un puntero.

- Se libera la memoria, pero el resto de los punteros sigue apuntando a la dirección.
- Genera errores de segmentación, o modificaciones no deseadas del programa (dirección reutilizada).

Dangling (dejar colgado)

Esta situación ocurre cuando un segmento de memoria es referenciado por más de un puntero.

- Se libera la memoria, pero el resto de los punteros sigue apuntando a la dirección.
- Genera errores de segmentación, o modificaciones no deseadas del programa (direccion reutilizada).

```
In [ ]: #include <iostream>

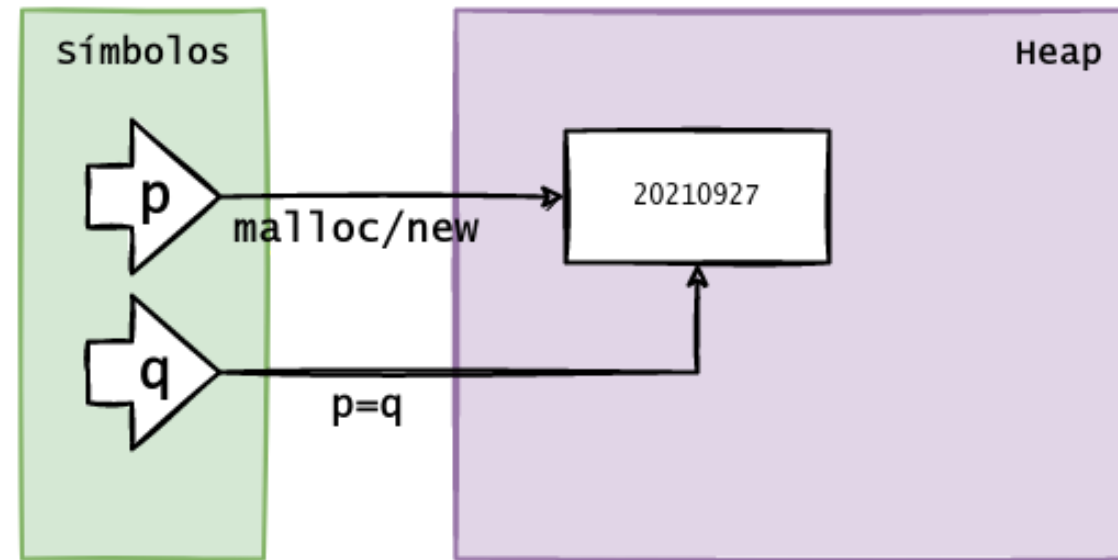
int main(){

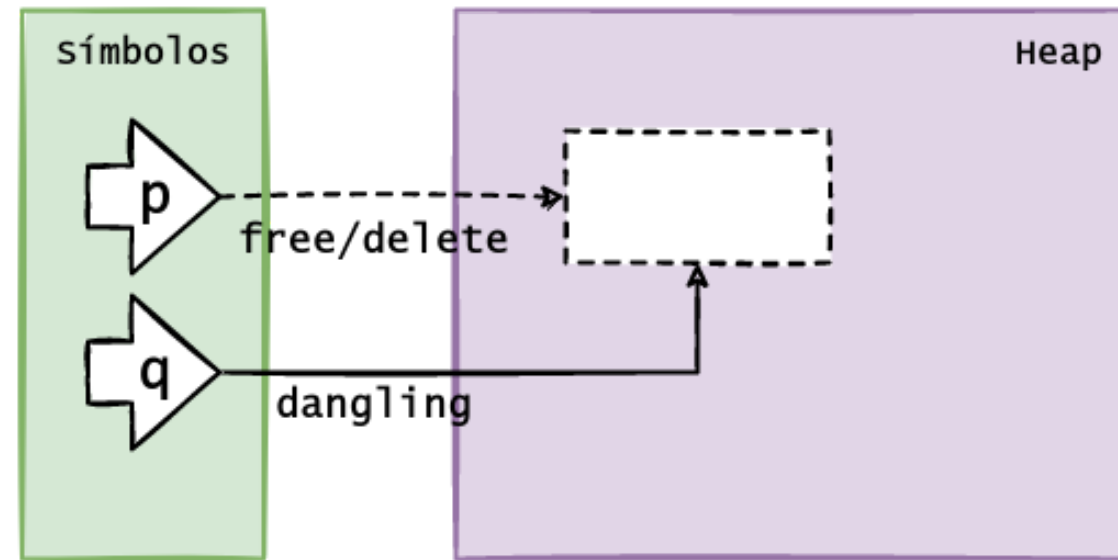
    int *p = new int;
    *p = 20210927;
    int *q = p;

    delete p;

    std::cout << "valor al que apunta q: " << *q << std::endl;

    return 0;
}
```





Valgrind

Valgrind

En lineas generales, estos problemas son dificiles de detectar y reparar.

- El compilador no tiene procesos depurados para identificarlos.
- El uso de varios punteros complica encontrar su origen.
- Se recomienda el uso de la herramienta **valgrind**, que nos ayuda a entender bien el origenes de los problemas de memoria.

